

Machine Learning Project Design Doc

Template for classical ML and deep learning projects: supervised, unsupervised, and predictive systems trained on your own data.

0. Project Summary

- **Problem:**
- **Why it matters / impact:**
- **ML task type:** (classification, regression, ranking, clustering, forecasting, etc.)
- **Proposed approach:**
- **Baseline to beat:**
- **Primary success metric:**
- **Key risks / unknowns:**

1. Problem Framing & Success Metrics

Business / User Problem

Describe the underlying problem in plain language. Who is the user? What decision or workflow does the model improve? Why is ML the right approach (vs. rules/heuristics, traditional software, or an LLM prompt)? What is the cost of a wrong prediction, and is it symmetric?

ML Problem Formulation

Translate the business problem into an ML task. Define the prediction target (label), the unit of prediction (per user, per document, per day), the prediction horizon, and when the prediction is made relative to when the label becomes known.

Aspect	Definition
Task type	
Target / label	
Unit of prediction	
Prediction time vs. label time	
Action taken on prediction	

Goal

Specific, measurable goal(s). What does "solved" look like? Tie to user outcomes and timelines.

Stakeholders

Key people/teams and their roles/ownership (PM, ML Eng, Data Eng, Domain Experts / Labelers, Legal, etc.).

Prior Work

Past attempts, existing heuristics, published approaches, benchmark datasets, Kaggle solutions. Include links.

Input and Output

What data goes into the model at training time vs. inference time, what the model outputs (class, score, probability, embedding), and how that output is consumed. What must be stored, and where.

Constraints

Constraint	Target	Notes
Inference latency (p50 / p99)	e.g., < 50ms online, batch overnight OK	
Throughput	e.g., 1k predictions/sec	
Training cost / compute	e.g., < 4 GPU-hours per run	
Model size / hardware	e.g., CPU-only, < 500MB	
Quality bar	e.g., F1 > 0.85, beats baseline by 10%	
Interpretability	e.g., per-prediction explanations required	
Privacy / compliance	e.g., no PII in features, GDPR/FERPA	
Fairness	e.g., metric parity within 5% across groups	

Success Metrics

- **Business / user metrics:** Revenue lift, time saved, error reduction, adoption, A/B test outcome.
- **Offline model metrics:** Accuracy, precision/recall/F1, ROC-AUC, PR-AUC, RMSE/MAE, NDCG, calibration (Brier, ECE). Choose metrics that reflect the cost asymmetry of errors.
- **System metrics:** Inference latency (p50/p99), throughput, training time, cost per prediction, uptime.

State explicitly how offline metrics are expected to map to the business metric, and how you will validate that mapping.

Checklist:

- ☐ I have strong evidence that the problem is important to solve.
- ☐ I have strong evidence that ML is the right tool (vs. rules, heuristics, or an LLM).
- ☐ I have a clear, unambiguous definition of the label and when it becomes available.
- ☐ My offline metric reflects the real cost of errors, and I know how it maps to the business goal.
- ☐ I have shared the scope with stakeholders and flagged dependencies.
- ☐ I have documented key decisions and trade-offs in my README or decision log.

2. Data Collection & Labeling

Data Sources

Source	Type	Volume	Update Freq.	Access / Owner	Notes

Labeling Strategy

How are labels obtained? (existing logs, human annotation, weak supervision, programmatic labeling, implicit feedback.) Who labels, with what guidelines? How is label quality measured (inter-annotator agreement, Cohen's kappa, audit sample)? What is the estimated label noise rate?

Aspect	Approach
Label source	
Annotation guidelines	Link to doc
# labeled examples	
Agreement / quality check	e.g., kappa on 10% double-labeled
Cost per label	

Data Versioning & Governance

How are datasets versioned (DVC, Delta Lake, dated snapshots)? Where is raw vs. processed data stored? Retention policy, PII handling, consent, and licensing of third-party data.

Checklist:

- ☐ I know the provenance and license of every data source.
- ☐ Labels have written guidelines and a measured quality/agreement score.
- ☐ Datasets are versioned so any experiment can be reproduced.
- ☐ PII and sensitive attributes are identified and handled per policy.

3. Exploratory Data Analysis & Data Quality

Summarize what you learned from the data before modeling.

Check	Finding	Action Taken
Class balance / target distribution		
Missing values		
Outliers / invalid values		
Duplicates / near-duplicates		
Feature correlations		
Distribution shift over time		
Potential leakage signals		

Checklist:

- ☐ I have profiled every feature and the target.
- ☐ I have explicitly checked for target leakage (features that encode the label or future information).
- ☐ I have documented data quality issues and how each was resolved.

4. Feature Engineering & Data Pipeline

Features

Feature	Source	Transformation	Available at Inference?	Notes

Pipeline

Describe preprocessing (imputation, scaling, encoding, tokenization, augmentation) and where it runs. The same transformation code must run at training and inference to avoid train/serve skew (e.g., scikit-learn Pipeline, feature store, shared transform module).

Checklist:

- ☐ Every feature is available at prediction time with the same latency and semantics as in training.
- ☐ Preprocessing is fit on training data only, then applied to validation/test.
- ☐ Training and serving share one transformation code path.
- ☐ I have measured the contribution of major feature groups (ablation or importance).

5. Dataset Splitting & Validation Strategy

Describe how data is split and why. The split must mirror how the model will be used in production.

Strategy	When to Use
Random / stratified	i.i.d. data, no time or group structure; stratify for imbalanced classes
Time-based (temporal)	Forecasting or any data where the future is predicted from the past
Grouped (GroupKFold)	Multiple rows per user/patient/document; prevents same entity in train and test
K-fold cross-validation	Small datasets; report mean and std across folds

Split	# Examples	Date Range / Criteria	Class Distribution
Train			
Validation			
Test (held out)			

Checklist:

- ☐ The split strategy matches the production setting (time, groups, distribution).
- ☐ The test set was held out and not used for any tuning or feature decisions.
- ☐ No entity or time-period leakage exists across splits.

6. Baselines & Model Selection

Baselines

Always establish baselines before complex models: (1) trivial baseline (majority class, mean, last value), (2) heuristic or rule-based baseline, (3) simple model (logistic/linear regression, decision tree).

Baseline	Description	Metric (Val)	Notes
Trivial			
Heuristic / rules			
Simple model			

Candidate Models

Model	Family	Val Metric	Train Time	Inference Latency	Size	Interpretability

Families to consider: linear models, tree ensembles (XGBoost, LightGBM, CatBoost), kNN/SVM, neural nets (MLP, CNN, RNN/Transformer), pretrained models + fine-tuning. For tabular data, gradient-boosted trees are usually the bar to beat.

Checklist:

- ☐ I have a trivial baseline and a simple-model baseline with recorded metrics.
- ☐ I have compared at least 3 model families on the same splits and metrics.
- ☐ I have justified any added complexity with a measurable gain over simpler models.
- ☐ I have documented the rationale for my final model choice, including cost and interpretability trade-offs.
- ☐ Licenses of pretrained weights and libraries fit my use case.

7. Training & Experiment Tracking

Training Setup

Aspect	Approach
Loss function	and why it fits the task/cost structure
Class imbalance handling	e.g., class weights, resampling, focal loss, threshold tuning
Regularization	e.g., L1/L2, dropout, early stopping
Hyperparameter search	e.g., random search, Optuna/Bayesian, # trials, search space
Compute	e.g., local CPU, Colab GPU, cloud instance
Reproducibility	Fixed seeds, pinned dependencies, data version recorded per run

Experiment Tracking

Every run logs: code commit, data version, hyperparameters, metrics, artifacts, and training time. Tooling: e.g., MLflow, Weights & Biases, Neptune, or a structured CSV/SQLite log at minimum.

Run ID	Model	Key Hyperparams	Data Version	Val Metric	Notes

Checklist:

- ☐ Every experiment is logged with code, data, config, and results.
- ☐ Any result can be reproduced from its run record.
- ☐ Hyperparameters were tuned on validation data only.
- ☐ I have learning curves showing whether the model is data-limited or capacity-limited.

8. Evaluation & Error Analysis

Final Evaluation

Metric	Baseline	Final Model (Test)	Confidence Interval

Report uncertainty (bootstrap CIs or CV std). For classifiers, include the confusion matrix and the chosen decision threshold with its justification. Check calibration if predicted probabilities drive decisions.

Slice Analysis & Fairness

Evaluate performance on meaningful subgroups (demographics, regions, data sources, rare classes, new vs. existing users). Large gaps between slices are bugs to investigate, not footnotes.

Slice	# Examples	Metric	Gap vs. Overall	Notes

Robustness

Test on noisy, out-of-distribution, or adversarial inputs relevant to your domain. What happens with missing features or unseen categories?

Error Analysis

Manually review a sample of errors (e.g., 50 to 100), categorize them, and quantify each category. Tie each category to a concrete fix (more data, new feature, label cleanup, model change).

Interpretability

Global: feature importance, partial dependence. Local: SHAP, LIME, counterfactuals. Do explanations make sense to domain experts?

Checklist:

- ☐ Final numbers come from a test set touched only once.
- ☐ I report confidence intervals, not just point estimates.
- ☐ I have evaluated performance across important slices and documented gaps.
- ☐ I have categorized errors and linked each category to an improvement.
- ☐ Model behavior has been sanity-checked by a domain expert.

9. Deployment & Serving

Serving Pattern

Aspect	Approach
Mode	Batch (scheduled scoring) vs. online (real-time API) vs. edge/on-device
Framework	e.g., FastAPI, BentoML, TorchServe, SageMaker
Model format	e.g., pickle/joblib, ONNX, TorchScript
Model registry	e.g., MLflow Model Registry; versioned, with stage (staging/prod)
Authentication / rate limiting	
Fallback behavior	What happens if the model or a feature is unavailable?

Rollout

Describe the release plan: shadow mode, canary, A/B test, human-in-the-loop review. Define rollback criteria before launch.

User Interface & Infrastructure

Aspect	Approach
UI	e.g., Streamlit, Gradio, React, or integration into an existing tool
Feedback mechanism	How users flag wrong predictions (these become new labels)
Hosting	e.g., AWS, GCP, Azure, on-prem
Containerization	e.g., Docker
CI/CD	e.g., GitHub Actions: tests, data validation, and model eval on push

Checklist:

- ☐ Model artifacts are versioned in a registry and linked to their training run.
- ☐ I have a defined rollout plan and rollback criteria.
- ☐ The application is containerized and CI runs tests on push.
- ☐ Users can give feedback, and that feedback is captured for relabeling.
- ☐ I have a live demo link or clear setup instructions in my README.

10. Monitoring & Maintenance

Component	Metrics to Track
Input data	Schema violations, missing rates, feature distribution drift (PSI, KS test)
Predictions	Prediction distribution drift, confidence distribution, class ratio
Model quality	Online accuracy once labels arrive, slice metrics, calibration
System	Latency, throughput, error rate, cost, uptime

Logging

At minimum, log per prediction: timestamp, model version, input features (or hash if sensitive), prediction, confidence, latency. Join with ground truth when it arrives.

Retraining Strategy

Scheduled (e.g., monthly) vs. triggered (drift or metric drop past a threshold). How is a retrained model validated against the current production model before promotion?

Checklist:

- ☐ I log every prediction with model version and inputs.
- ☐ I monitor data drift and prediction drift with defined thresholds.
- ☐ I have alerting for metric regressions.
- ☐ I have a documented retraining and promotion process.

11. Responsible ML

Concern	Approach
Fairness	Which groups could be harmed? Which fairness metric and why?
Privacy	PII minimization, anonymization, access controls, right to deletion
Transparency	Model card documenting intended use, limits, and evaluation
Misuse / failure modes	Worst-case impact of a wrong prediction; human oversight
Feedback loops	Can the model's predictions bias its own future training data?

Checklist:

- ☐ I have written a model card.
- ☐ I have assessed fairness across relevant groups.
- ☐ High-stakes decisions have human review.

12. Code Quality & Repository Structure

Project Structure

```
your-project/
|-- src/
|   |-- data/           # Ingestion, validation, splitting
|   |-- features/       # Feature engineering, shared train/serve transforms
|   |-- models/         # Model definitions, training loops
|   |-- evaluation/     # Metrics, slice analysis, error analysis
|   |-- monitoring/     # Drift detection, logging
|-- configs/            # Hyperparameters and experiment configs (YAML)
|-- notebooks/         # EDA only; logic lives in src/
|-- api/               # Serving application
|-- frontend/          # UI code
|-- tests/             # Unit tests, data validation tests
|-- data/              # Sample data or DVC pointers (no raw data in git)
|-- models/            # Registry pointers or small artifacts
|-- deployment/        # Docker, cloud config
|-- docs/              # Architecture diagram, decision log, model card
|-- .env.example
|-- requirements.txt
|-- Dockerfile
`-- README.md
```

Code Standards

- ☐ Type hints and docstrings on all functions.
- ☐ No hardcoded values: paths, hyperparameters, and keys in config files or env vars.
- ☐ Notebooks for exploration only; reusable logic in .py modules.
- ☐ Unit tests for data transforms and data validation checks (e.g., Pandera, Great Expectations).
- ☐ Random seeds set and dependencies pinned.

README Contents

- ☐ Problem, ML formulation, and solution overview.
- ☐ Architecture diagram (data to training to serving to monitoring).
- ☐ Dataset description and how to obtain it.
- ☐ Baselines vs. final model results with confidence intervals.
- ☐ Key decisions and trade-offs.
- ☐ Setup instructions (clone, train, evaluate, serve).
- ☐ Live demo link (if deployed).

13. Project Timeline & Milestones

Milestone	Target Date	Status
Problem scoping & ML formulation		
Data collection & labeling		
EDA & data quality complete		
Baselines established		
Feature pipeline v1		
Model experiments & tuning		
Evaluation & error analysis		
Deployment & UI		
Monitoring & retraining pipeline		
Model card, README, demo		

14. Appendix

- Links to repos, notebooks, experiment dashboards, datasets.
- Architecture diagram(s).
- Model card.
- Glossary of terms and metrics.
- Decision log: key choices made and why.